

eCryptfs Custom ACL

Kernel-Level Access Control Implementation

Project Implementation Report

Presented by: **Frank**

Supervised by: **Prof. Yang**

Technical Stack: Linux 5.15 | Out-of-tree Module | Kernel C

Date: March 2026

Big Picture

- **Problem** — eCryptfs encrypts files but has no control over *who reads what*
- **Solution** — Kernel-level ACL that decides not just allow/deny, but *what data is returned*
- **Key result** — Same file → different data per process, enforced in kernel space

★ Two processes can both be *allowed* to open the same file yet receive completely different bytes — one gets plaintext, the other raw ciphertext.

| **01**

| **Background & Objective**

What is eCryptfs?

eCryptfs is a **stacked cryptographic filesystem** built into the Linux kernel.

- Sits between the VFS layer and a lower filesystem (ext4, etc.)
- Transparently **encrypts files** on write, **decrypts** on read
- Each file carries its own encryption metadata in the header
- Mainline Linux since 2.6.19 · secure storage · embedded systems

```
User process
  ↓
VFS layer
  ↓
eCryptfs ← intercepts read/write, encrypts/decrypts
  ↓
Lower FS (ext4) – stores raw ciphertext
```

The Problem

eCryptfs has **no fine-grained access control** beyond standard POSIX permissions.

Limitation	Impact
Any permitted process reads plaintext	No isolation between processes
No per-process content control	Cannot serve ciphertext to one, plaintext to another
No identity-based rules	Cannot restrict by UID, GID, or executable
Single permission model	Cannot have read-only ciphertext access

“ **Need:** A second enforcement layer — controls **who gets what data**, not just whether access is allowed. ”

Project Objective

Design and implement an **internal ACL system** for a custom eCryptfs module that:

- Runs **after** standard POSIX/VFS permissions (two-stage model)
- Controls access based on **UID + GID + process executable** (AND logic)
- Decides not just **allow/deny** but also **what data is returned**:
 - `plaintext` — decrypted content
 - `ciphertext` — raw encrypted bytes
 - `deny` — access refused
- Enforced entirely in **kernel space** — no LSM dependency
- Lightweight: 2-byte xattr index, ≤ 64 rules per file

| 02

| Key Contributions

Key Contributions

Three deliverables, each independently verifiable:

#	Deliverable	What it shows
1	Requirements & Design Document	Problem understanding; method selection
2	Kernel Implementation	~1,600 lines across 6 files; working module
3	Automated Test Suite	32 tests; every ACL dimension verified

1 — Requirements & Design Document

`doc/acl_srs_en.md` · v6.0 · 18 sections

Key decisions made during design:

- **Access model:** three-dimensional (Subject × Permission × Content Mode) — beyond simple allow/deny
- **Process identity:** match by dev+inode number, resolved at rule-insertion time — stable and fast
- **Dual page cache:** two independent `address_space` per inode — no cross-contamination between plaintext and ciphertext readers
- **Inheritance:** dynamic upward walk at access time — xattr present = explicit override, absent = inherit (unambiguous by construction)

2 — Kernel Implementation

New files

- `acl.h` — data structures, constants, API
- `acl.c` — ACL engine, dual cache, debugfs

Modified files

- `ecryptfs_kernel.h` — inode info fields
- `inode.c` — permission check, inheritance
- `file.c` — content mode, ciphertext read path
- `super.c` — inode lifecycle hooks

Environment

- Kernel: Linux 5.15.196 (out-of-tree module)
- Build: `make` against kernel headers
- Load: `insmod ecryptfs.ko`
- Manage: debugfs interface

~**1,600 lines** added across 6 files

3 — Automated Test Suite

`tests/` · Python + pytest · **32 tests** · **all passing**

- **Black-box behavioural:** real binaries, real mounted files, real users
- `fork + setuid + exec` — kernel sees authentic uid, gid, and exe path

Covers: UID / GID / executable matching · plaintext / ciphertext / deny outcomes · priority ordering · directory inheritance · permission bits (r / w / rw)

| 03

| Design

Access Model

Every access decision has **three dimensions**:

- **Subject** — WHO: uid + gid + process executable (AND logic)
- **Permission** — WHAT operations: r / w / x (intersected with system permissions)
- **Content Mode** — WHAT DATA is returned: `plaintext` | `ciphertext` | `deny`

Standard ACLs control only allow/deny. This system also controls **what bytes the process receives** — two allowed processes can get different data from the same file.

Rule Matching Logic

Subject matching — AND logic

All specified conditions must match:

Field	Value	Meaning
uid	number or *	User ID
gid	number or *	Group ID
exe	path or *	Executable

* = wildcard (skip this dimension)

Decision — first-match + priority

Rules sorted by priority ↓

```
prio=100 uid=1001 exe=head
  ↓ matches? → apply, stop
prio=50  uid=*   exe=cat
  ↓ matches? → apply, stop
(no match) → DENY
```

Max **64 rules** per ACL ID.

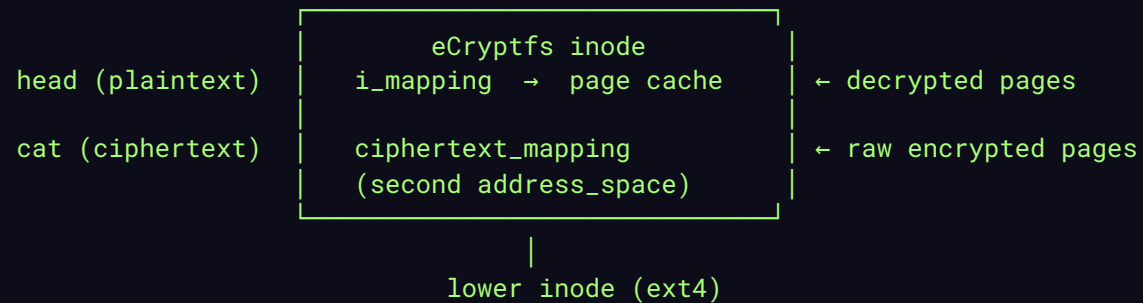
Three-Point Enforcement

```
Process opens file
|
▼
POINT 1 – ecryptfs_permission()      [inode.c]
  Stage 1: system POSIX check
  Stage 2: ACL allow / deny gate
  |
  ▼ (allowed)
POINT 2 – ecryptfs_open()            [file.c]
  Decide content mode → store per-fd
  |
  ▼
POINT 3 – ecryptfs_read()            [file.c]
  PLAINTEXT → decrypt from page cache
  CIPHERTEXT → read raw bytes from lower file
```

“ Three points are needed: content mode requires per-fd state; ciphertext must bypass the eCryptfs page cache entirely. ”

Dual Address Space ★

The same inode serves **different data** to different processes simultaneously.



Two independent page caches — no cross-contamination. This is the core innovation.

Storage Model

- **Per-file:** 2-byte xattr `trusted.ecryptfs_acl_id` on the lower inode
- **In kernel:** per-mount hash table maps ACL ID → sorted rule list (max 64 rules)
- **Inheritance:** no xattr on file → walk up dentry tree to nearest tagged ancestor

```
acl_table (per mount)
└─ ecryptfs_acl_entry (acl_id=1)
    └─ rule[0] prio=100 exe=cat → ciphertext
    └─ rule[1] prio=50  exe=head → plaintext
    └─ ...
```

“ xattr stores only a 2-byte index — fast lookup, no per-access xattr reads after first resolve. ”

| 04

| Implementation

Implementation Overview

Four phases, all complete:

Phase	Deliverable
1	Data structures, xattr read, hash table, module init
2	Debugfs management interface (<code>add</code> / <code>clear</code> / <code>list</code>)
3	Process matching (dev+ino), ciphertext read path, permission enforcement
4	Directory inheritance — upward dentry walk to mount root

Key Decisions

Decision	Why
<code>vfs_iter_read(lower_file)</code> for ciphertext	<code>generic_file_read_iter</code> has a size-bound bug through the eCryptfs page cache
Process match via dev + inode (not path)	Path is namespace-dependent; inode number is stable
Resolve inode at rule-insertion time	Avoids <code>kern_path()</code> lookup on every file access
No xattr → pass-through	Backward compatible with existing unprotected files

Rule Management

Rule format

```
add <acl_id> <priority> <uid> <gid> <proc> <perm> <content>
```

Rules are managed via a debugfs control file:

```
# Add a rule  
echo "add 1 100 * * /usr/bin/head r plaintext" \  
| sudo tee /sys/kernel/debug/ecryptfs_acl/0/control  
  
# View rules  
sudo cat /sys/kernel/debug/ecryptfs_acl/0/rules
```

Tag a file or directory:

```
# Tag a directory – every file inside inherits the rules  
setfattr -n trusted.ecryptfs_acl_id -v 0x0001 /data.enc/private/
```

Directory Inheritance

A file with **no xattr** inherits the ACL from the nearest tagged ancestor.

```
/data.enc/  
  private/      ← acl_id = 0x0001  
    report.pdf  ← no xattr → inherits ACL 1  
    sub/  
      notes.txt ← no xattr → inherits ACL 1 (grandparent)
```

- Walk is **dynamic** — resolved lazily on first access, then cached per-inode
- Stops at the eCryptfs mount root; capped at 32 levels

| 05

| Testing

Testing

32 automated tests · Python + pytest · all passing

- **Black-box:** real binaries (`cat` , `head` , `dd`) run against real mounted encrypted files
- **Real identity:** `fork + setuid + exec` — kernel sees authentic uid, gid, and exe path

Covers:

- UID / GID / executable matching and AND logic
- All three outcomes: plaintext · ciphertext · deny
- Priority ordering · directory inheritance · permission bits (r / w / rw)

Test Results

```
tests/test_uid.py::test_matching_uid_plaintext
  scenario : uid=1001 + exe=head → plaintext
  result   : exit=0  PASSED

tests/test_inheritance.py::test_nested_inherits_plaintext
  scenario : file two levels below tagged dir → inherits plaintext
  result   : exit=0  PASSED

tests/test_permissions.py::test_perm_r_denies_write
  scenario : perm=r, dd tries to write → denied
  result   : exit=1  PASSED
```

Manual Demo

Tag a file and add rules:

```
setfattr -n trusted.ecryptfs_acl_id -v 0x0001 /testbase/data.enc/demo.txt
echo "add 1 100 * * /usr/bin/head r plaintext" | sudo tee /sys/kernel/debug/ecryptfs_acl/0/control
echo "add 1 50 * * /usr/bin/cat r ciphertext" | sudo tee /sys/kernel/debug/ecryptfs_acl/0/control
```

Three processes, same file, three different outcomes:

```
head /testbase/data.view/demo.txt # → decrypted plaintext
cat /testbase/data.view/demo.txt  # → raw ciphertext bytes
tail /testbase/data.view/demo.txt # → EACCES (no matching rule)
```

| 06

| **Status & Next Steps**

Current Status

✓ Completed

- ACL data structures & hash table
- xattr read (`trusted.ecryptfs_acl_id`)
- Debugfs management interface
- UID / GID / exe subject matching
- Three-point kernel enforcement
- Dual address_space (cache isolation)
- Ciphertext read path
- Directory inheritance
- 32 automated tests — all passing

→ SOON Next Steps

- **Persistence**
Save/load rules across reboot
- **Stronger process matching**
SHA-256 of executable binary
- **Stale inode re-resolve**
Auto-recover when binary is upgraded
- **Userspace CLI**
Management tool beyond debugfs

Summary

Built	Custom ACL layer inside eCryptfs kernel module
Access model	Subject × Permission × Content Mode
Core innovation	Same inode → different data per process (dual address_space)
Enforcement	Three kernel hook points — permission, open, read
Validated	32 automated tests — all passing
Codebase	~1,600 lines across 6 kernel source files

“ Core enforcement is complete and verified. Next steps are additive — the foundation is solid. ”

Thank You

Repository: `github.com/netcaf/myecryptfs`

Design document: `doc/acl_srs_en.md`

Run tests:

```
sudo pytest tests/ -v
```